Comparison of SGD With Momentum vs Adam Optimizer

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# Step 2: Set Device and Load Dataset
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# AI23531 Deep Learning Page | 16
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
transform=transform, download=True)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=True)

# Step 3: Define MLP Model
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.net = nn.Sequential(
            nn.Linear(3*32*32, 256),
            nn.ReLU(),
            nn.Linear(256, 10)
        )
    def forward(self, x):
        x = self.flatten(x)
        return self.net(x)

# Step 4: Define Training Function
def train(model, optimizer, epochs=10):
    model.to(device)
    loss_fn = nn.CrossEntropyLoss()
    losses = []
    accuracies = []
```

```
    for epoch in range(epochs):
        total_loss = 0
        correct = 0
        total = 0
        model.train()
        for imgs, labels in trainloader:
            imgs, labels = imgs.to(device), labels.to(device)
            outputs = model(imgs)
            loss = loss_fn(outputs, labels)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

        # AI23531 Deep Learning Page | 17
        avg_loss = total_loss / len(trainloader)
        accuracy = 100.0 * correct / total

        losses.append(avg_loss)
        accuracies.append(accuracy)
        print(f"Epoch {epoch+1}: Loss = {avg_loss:.4f}, Accuracy =
{accuracy:.2f}%")
    return losses, accuracies

# Step 5: Train with SGD + Momentum and with Adam
model_sgd = MLP()
sgd = optim.SGD(model_sgd.parameters(), lr=0.01, momentum=0.9)
losses_sgd, acc_sgd = train(model_sgd, sgd)

model_adam = MLP()
adam = optim.Adam(model_adam.parameters(), lr=0.001)
losses_adam, acc_adam = train(model_adam, adam)

# Step 6: Visualize Loss and Accuracy Comparison
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(losses_sgd, label="SGD + Momentum")
plt.plot(losses_adam, label="Adam")
plt.xlabel("Epoch")
plt.ylabel("Loss")
```

```
plt.title("Loss Comparison on CIFAR-10 (MLP)")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(acc_sgd, label="SGD + Momentum")
plt.plot(acc_adam, label="Adam")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("Accuracy Comparison on CIFAR-10 (MLP)")
plt.legend()
plt.tight_layout()
plt.show()
```

OUTPUT:

```
100%|██████████| 170M/170M [18:53<00:00, 150kB/s]

SGD Epoch 1 Loss: 1.79 Accuracy: 37.0 %
SGD Epoch 2 Loss: 1.5592 Accuracy: 45.5 %
SGD Epoch 3 Loss: 1.4675 Accuracy: 48.78 %
Adam Epoch 1 Loss: 1.6475 Accuracy: 41.9 %
Adam Epoch 2 Loss: 1.4437 Accuracy: 49.68 %
Adam Epoch 3 Loss: 1.3453 Accuracy: 53.24 %
```

